MACHINE LEARNING AND APPLICATIONS

[UE21EC352B]

"Automated Blood Cell Detection and Classification using Machine Learning for Complete Blood Count Analysis"

Submitted By :-

Dhanush D [PES1UG21EC082]

Darshan Raj G [PES1UG21EC079]

Under the guidance of

Dr. H R Vanamala

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

PES University Bengaluru - 560085

CONTENTS :-

1) Abstract.

2) Introduction.

3) Dataset.

4) Work Done.

5) Code.

6) Results.

7) Conclusion.

8) References.

Abstract :-

A complete blood cell count is an important test in medical diagnosis to evaluate overall health condition. Traditionally blood cells are counted manually using haemocytometer along with other laboratory equipment's and chemical compounds, which is a time-consuming and tedious task. In this work, we present a machine learning approach for automatic identification and counting of three types of blood cells using 'you only look once' (YOLO) object detection and classification algorithm. YOLO framework has been trained with a modified configuration BCCD Dataset of blood smear images to automatically identify and count red blood cells, white blood cells, and platelets. Moreover, this study with other convolutional neural network architectures considering architecture complexity, reported accuracy, and running time with this framework and compare the accuracy of the models for blood cells detection.

Introduction :-

A complete blood cell (CBC) count is an important test often requested by medical professionals to evaluate health condition. As these blood cells are huge in number, traditional manual blood cell counting system using haemocytometer is highly time consuming an erroneous and most of the cases accuracy vastly depends on the skills of a clinical laboratory analyst . In general, there are generally two different approaches in the automated counting process of blood cells. Therefore, an automated process to count different blood cells from a smear image will greatly facilitate the entire counting process with the development of machine learning techniques, image classification and object detection applications are becoming more robust and more accurate. A DEEP learning based blood cell counting method has been proposed. We employed a deep learning based object detection method to detect different blood cells. Among the state-of-the-arts object detection algorithms such as regions with convolutional neural network (R-CNN) , you only look once(YOLO) , we chose YOLO framework which is about three times faster than Faster R-CNN with VGG-16 architecture. YOLO uses a single neural network to predict bounding boxes and class probabilities directly from the full image in one evaluation. We train YOLO framework to automatically identify and count RBCs, WBCs, and platelets from blood smear images.

Dataset :-

Blood Cell Count Dataset (BCCD){https://github.com/Shenggan/BCCD_Dataset}. Total of 360 blood smear images. After splitting the dataset into training (330) and testing (30) parts.

Splitting ratio: 70:20:10.

Work Done :-

Our code is designed to automatically identify and count blood cells. YOLO: 'You Only Look Once' in short YOLO is a state-of-the-art object detection classification algorithm .It treats object detection as a regression problem. It requires only one forward propagation pass through the network to make a fast prediction for both image class and location. It resizes the image by 448 × 448 and divides the entire image into a 7 × 7 grid cell and each grid cell predicts for two bounding boxes and confidence score for the boxes. If the center of the object falls into a grid cell that the grid cell is responsible for detecting that object. Its network architecture contains 24 convolutional layers and 2 fully connected layers and inspired by the GoogLeNet. Among different versions of it, we choose to use YOLOv5 as it is the fastest. YOLOv5 uses 9 instead of 24 convolutional layers other than that all the parameters are the same.

Code :-

```
# clone YOLOv5 repository
!git clone https://github.com/ultralytics/yolov5  # clone repo
%cd yolov5
!git reset --hard 064365d8683fd002e9ad789c1e91fa3d021b44f0
```

```
[ ] # install dependencies as necessary
    !pip install -qr requirements.txt  # install dependencies (ignore errors)
    import torch

    from IPython.display import Image, clear_output  # to display images
    from utils.downloads import attempt_download  # to download models/datasets

    # clear_output()
    print('Setup complete. Using torch %s %s' % (torch.__version__, torch.cuda.get_device_properties(0) if torch.cuda.is_available() else 'CPU'))
```

```
[ ] !pip install roboflow

    from roboflow import Roboflow
    rf = Roboflow(api_key="SZnQc08FwM271feVRMhd")
    project = rf.workspace("raj-a7ysh").project("rajanna_train_yolov5")
    version = project.version(4)
    dataset = version.download("yolov5")
```

```
[ ] %cd /content/yolov5
```

```
[ ] # this is the YAML file Roboflow wrote for us that we're loading into this notebook with our data
    %cat /content/yolov5/rajanna_train_yolov5-4/data.yaml
```

```
[ ] # define number of classes based on YAML
    import yaml
    with open("/content/yolov5/rajanna_train_yolov5-4/data.yaml", 'r') as stream:
        num_classes = str(yaml.safe_load(stream)['nc'])
```

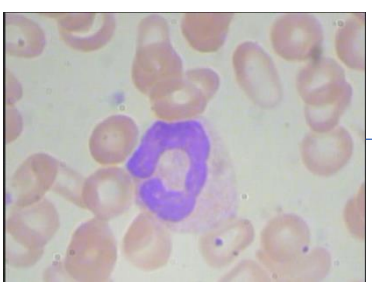
```
[ ] #this is the model configuration we will use for our tutorial
    %cat /content/yolov5/models/yolov5s.yaml
```

```
[ ] #customize iPython writefile so we can write variables
    from IPython.core.magic import register_line_cell_magic

    @register_line_cell_magic
    def writetemplate(line, cell):
        with open(line, 'w') as f:
            f.write(cell.format(**globals()))
```

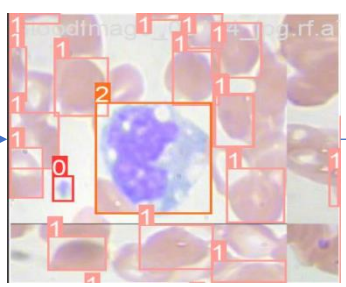
```
%%writetemplate /content/yolov5/models/custom_yolov5s.yaml

# parameters
nc: {num_classes}  # number of classes
depth_multiple: 0.33  # model depth multiple
width_multiple: 0.50  # layer channel multiple

# anchors
anchors:
  - [10,13, 16,30, 33,23]  # P3/8
  - [30,61, 62,45, 59,119]  # P4/16
  - [116,90, 156,198, 373,326]  # P5/32

# YOLOv5 backbone
backbone:
  # [from, number, module, args]
  [[-1, 1, Focus, [64, 3]],  # 0-P1/2
   [-1, 1, Conv, [128, 3, 2]],  # 1-P2/4
   [-1, 3, BottleneckCSP, [128]],
   [-1, 1, Conv, [256, 3, 2]],  # 3-P3/8
   [-1, 9, BottleneckCSP, [256]],
   [-1, 1, Conv, [512, 3, 2]],  # 5-P4/16
   [-1, 9, BottleneckCSP, [512]],
   [-1, 1, Conv, [1024, 3, 2]],  # 7-P5/32
   [-1, 1, SPP, [1024, [5, 9, 13]]],
   [-1, 3, BottleneckCSP, [1024, False]],  # 9
  ]

# YOLOv5 head
head:
  [[-1, 1, Conv, [512, 1, 1]],
   [-1, 1, nn.Upsample, [None, 2, 'nearest']],
   [[-1, 6], 1, Concat, [1]],  # cat backbone P4
   [-1, 3, BottleneckCSP, [512, False]],  # 13

   [-1, 1, Conv, [256, 1, 1]],
   [-1, 1, nn.Upsample, [None, 2, 'nearest']],
   [[-1, 4], 1, Concat, [1]],  # cat backbone P3
   [-1, 3, BottleneckCSP, [256, False]],  # 17 (P3/8-small)

   [-1, 1, Conv, [256, 3, 2]],
   [[-1, 14], 1, Concat, [1]],  # cat head P4
   [-1, 3, BottleneckCSP, [512, False]],  # 20 (P4/16-medium)

   [-1, 1, Conv, [512, 3, 2]],
   [[-1, 10], 1, Concat, [1]],  # cat head P5
   [-1, 3, BottleneckCSP, [1024, False]],  # 23 (P5/32-large)

   [[17, 20, 23], 1, Detect, [nc, anchors]],  # Detect(P3, P4, P5)
```

```
[ ] # train yolov5s on custom data for 100 epochs
    # time its performance
    %%time
    %cd /content/yolov5/
    !python train.py --img 416 --batch 16 --epochs 100 --data /content/yolov5/rajanna_train_yolov5-4/data.yaml --cfg ./models/custom_yolov5s.yaml --weights '' --name yolov5s_results  --cache

[ ] from utils.plots import plot_results  # plot results.txt as results.png
    from IPython.display import Image
    Image(filename='/content/yolov5/runs/train/yolov5s_results/results.png', width=1000)  # view results.png

[ ] Image(filename='/content/yolov5/runs/train/yolov5s_results/val_batch0_labels.jpg', width=900)

[ ] # print out an augmented training example
    print("GROUND TRUTH AUGMENTED TRAINING DATA:")
    Image(filename='/content/yolov5/runs/train/yolov5s_results/train_batch0.jpg', width=900)

[ ] # trained weights are saved by default in our weights folder
    %ls runs/

[ ] %ls runs/train/yolov5s_results/weights

    %cd /content/yolov5/
    !python detect.py --weights runs/train/yolov5s_results/weights/best.pt --img 416 --conf 0.4 --source /content/yolov5/rajanna_train_yolov5-4/test/images/

[ ] import glob
    from IPython.display import Image, display

    for imageName in glob.glob('/content/yolov5/runs/detect/exp/*.jpg')[:10]: #assuming JPG
        display(Image(filename=imageName))
```

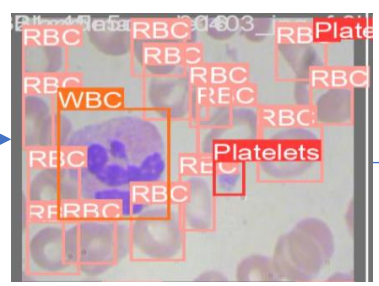
Results:-



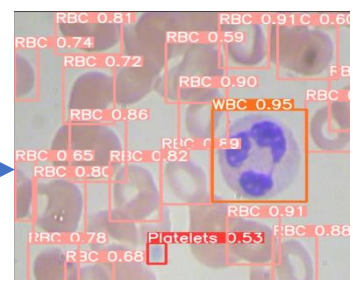
Input image



Ground truth augmented image



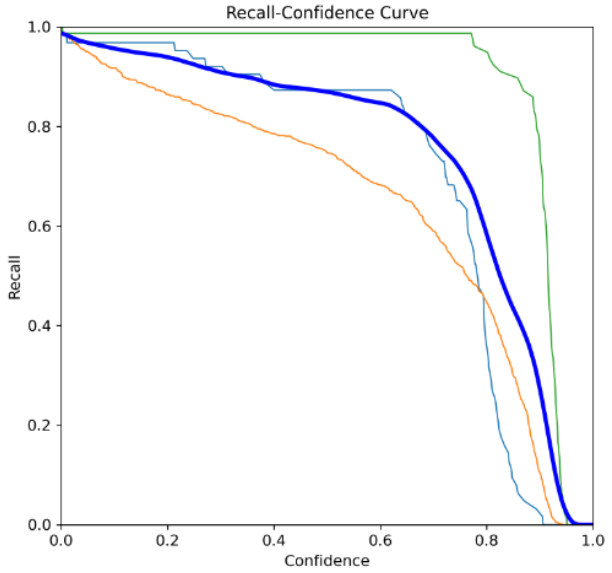
Validated image

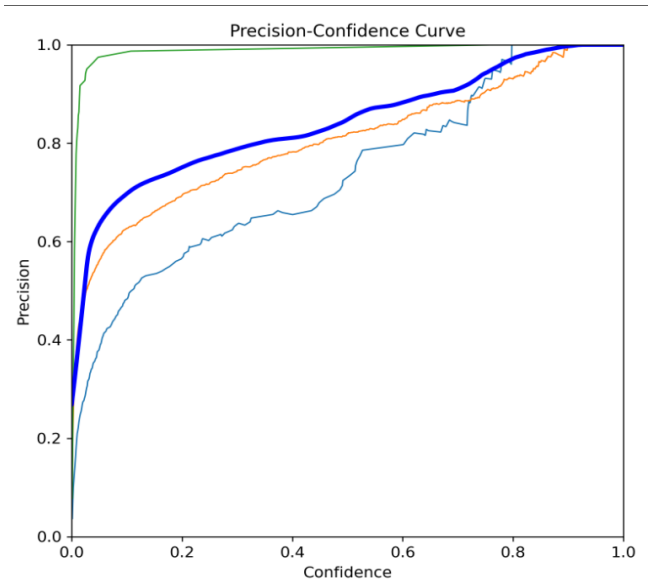


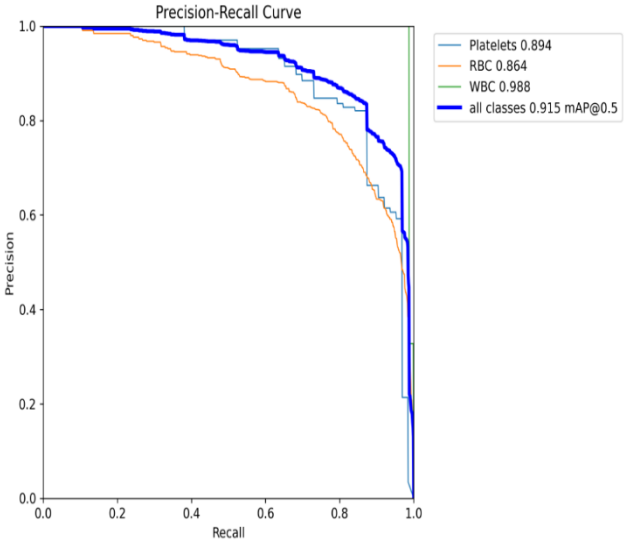
Test image

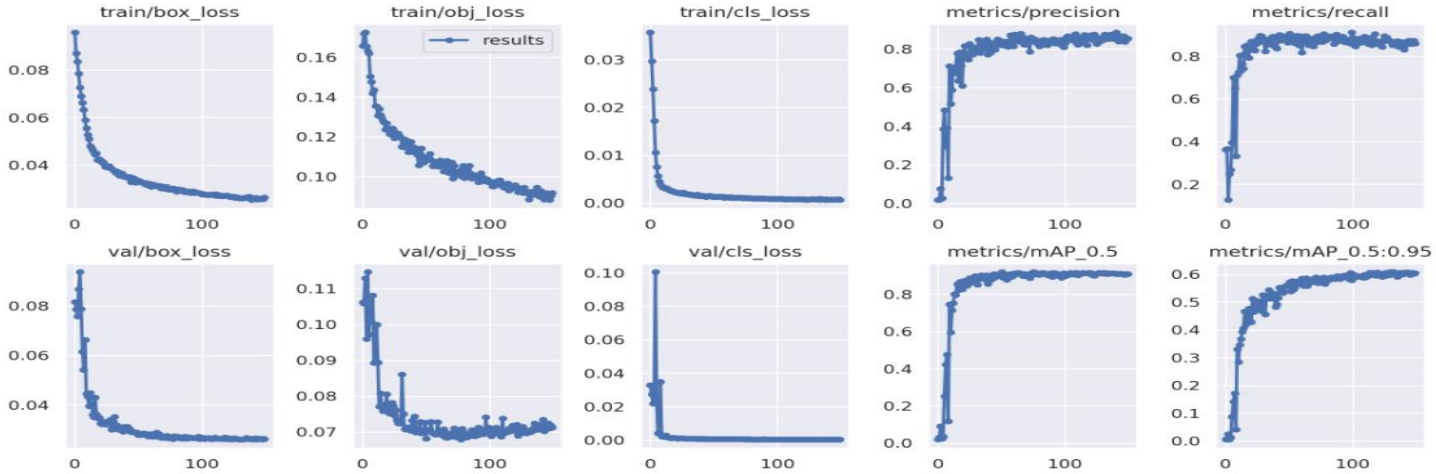
```
custom_YOLOv5s summary: 182 layers, 7251912 parameters, 0 gradients
                 Class     Images  Instances          P          R      mAP50   mAP50-95: 100% 3/3 [00:05<00:00,
                   all         75        952      0.876      0.852      0.915      0.608
             Platelets         75         63      0.793      0.873      0.894      0.455
                   RBC         75        811      0.839      0.695      0.864      0.603
                   WBC         75         78      0.996      0.987      0.988      0.767
Results saved to runs/train/yolov5s_results
CPU times: user 18.5 s, sys: 1.96 s, total: 20.4 s
Wall time: 27min 7s
```

- Summary of YOLOv5 with various Metrics


Recall-Confidence Curve
Recall
Confidence


Precision-Confidence Curve
Precision
Confidence


Precision-Recall Curve
Platelets 0.894
RBC 0.864
WBC 0.988
all classes 0.915 mAP@0.5
Precision
Recall


train/box_loss
train/obj_loss
results
train/cls_loss
metrics/precision
metrics/recall
val/box_loss
val/obj_loss
val/cls_loss
metrics/mAP_0.5
metrics/mAP_0.5:0.95

```
Fusing layers...
custom_YOLOv5s summary: 182 layers, 7251912 parameters, 0 gradients
image 1/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00007_jpg.rf.b2a38833e963c10c8dd87742cf991bb5.jpg: 416x416 1 Platelets, 18 RBCs, 1 WBC, 8.5ms
image 2/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00033_jpg.rf.7cc95c0d3b4afe7d8ea484e615f49515.jpg: 416x416 17 RBCs, 1 WBC, 7.7ms
image 3/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00036_jpg.rf.e60ff34dc039a0fab62fbd13e6f2b704.jpg: 416x416 1 Platelets, 26 RBCs, 1 WBC, 8.2ms
image 4/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00037_jpg.rf.a321a0adc4e74517f2b6181a75203aed.jpg: 416x416 2 Plateletss, 26 RBCs, 1 WBC, 7.8ms
image 5/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00043_jpg.rf.7e6ffe5d8dfd29588943c93d475af611.jpg: 416x416 2 Plateletss, 14 RBCs, 2 WBCs, 7.7ms
image 6/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00047_jpg.rf.7e0efda5aca0eab01f3d80de0317c985.jpg: 416x416 3 Plateletss, 16 RBCs, 2 WBCs, 7.7ms
image 7/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00049_jpg.rf.252548a62b50677bf015de938519faf0.jpg: 416x416 1 Platelets, 15 RBCs, 1 WBC, 7.7ms
image 8/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00072_jpg.rf.b00e000574df97f0e732ca73c7dc0e1b.jpg: 416x416 14 RBCs, 1 WBC, 7.7ms
image 9/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00109_jpg.rf.0fda5572c42c1250c28e86fdcc96f28f.jpg: 416x416 13 RBCs, 1 WBC, 8.8ms
image 10/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00134_jpg.rf.9783a97cd335ae88f3ddb8d31082432f.jpg: 416x416 3 Plateletss, 10 RBCs, 2 WBCs, 7.7ms
image 11/38 /content/yolov5/rajanna_train_yolov5-4/test/images/BloodImage_00141_jpg.rf.4a69f326823ca082e4847ddc020a6076.jpg: 416x416 15 RBCs, 1 WBC, 9.8ms
```

Conclusion:- A machine learning approach to automatically identify and count blood cells from a smear image based on YOLOv5 algorithm is presented. Our proposed method is evaluated on publicly available datasets. It is observed for test dataset that, our method accurately identifies RBCs, WBCs, and Platelets, takes less execution time. It is seen that our method can accurately count even some of the cells that are not labelled in the dataset. Our proposed method is much faster than the existing architectures. With the accuracy and the detection performance of the proposed method, it can be said that, the method has the potential to ease up the manual blood cell identification and counting process.

References:-

[1] D. Trong Luong, D. Duy Anh, T. Xuan Thang, H. Thi Lan Huong, T. Thuy Hanh and D. Minh Khanh, "Distinguish normal white blood cells from leukemia cells by detection, classification, and counting blood cells using YOLOv5," *2022 7th National Scientific Conference on Applying New Technology in Green Buildings (ATiGB)*, Da Nang, Vietnam, 2022, pp. 156-160, doi: 10.1109/ATiGB56486.2022.9984098.

[2] V. B. Inchur, L. S. Praveen and P. Shankpal, "Implementation of Blood Cell Counting Algorithm using Digital Image Processing Techniques," *2020 International Conference on Recent Trends on Electronics, Information, Communication & Technology (RTEICT)*, Bangalore, India, 2020, pp. 21-26, doi: 10.1109/RTEICT49044.2020.9315603.